Assignment-13.3

Hithesh

2303A51291

Batch-05

Task-01:

Prompt:

Refactor the following legacy Python script to remove code duplication and improve maintainability.

use coding practices use functions and required parameters.

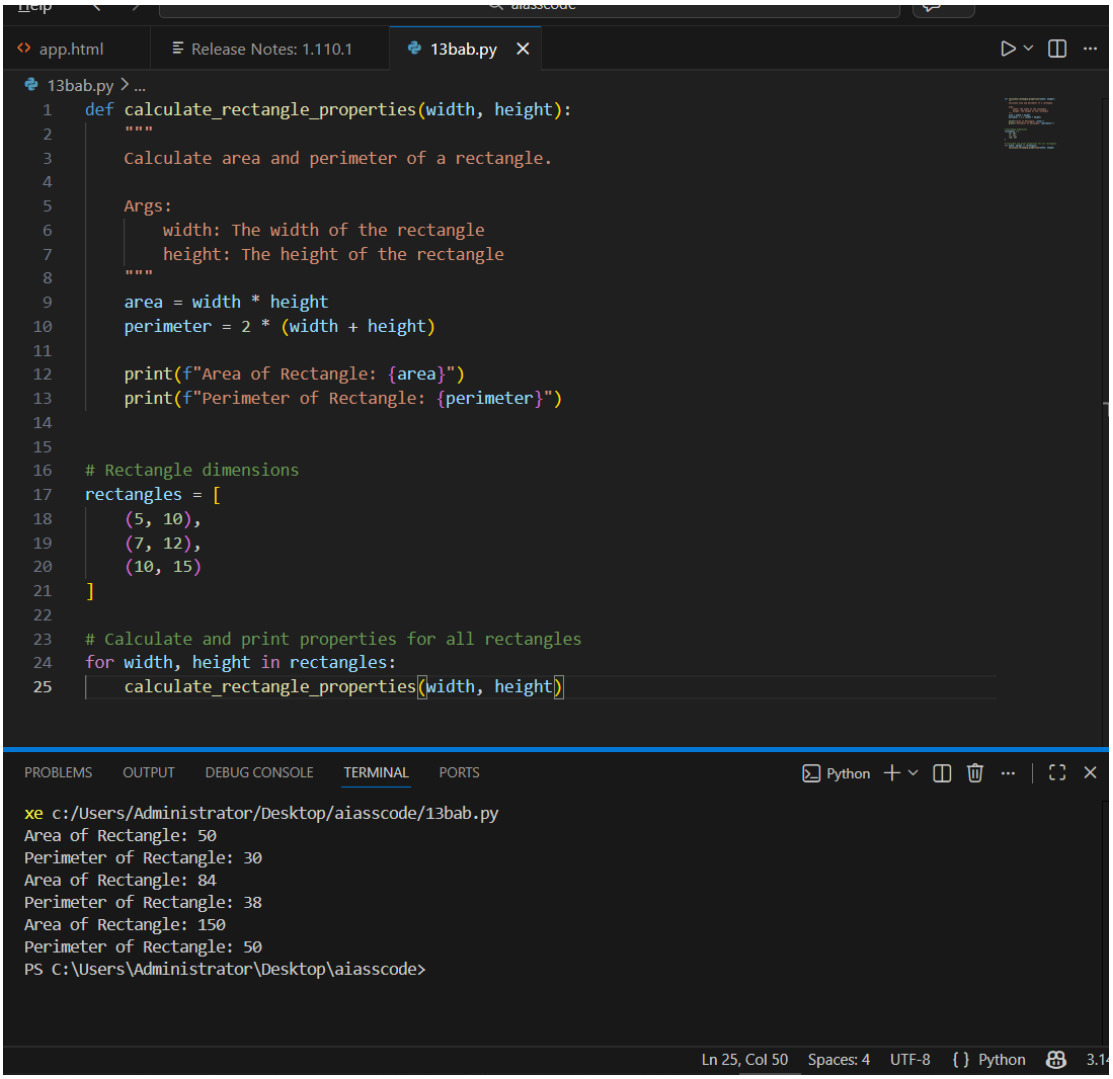
The program should still print the area and perimeter for all given rectangles.

Legacy Code:

```
print("Area of Rectangle:", 5 * 10)

print("Perimeter of Rectangle:", 2 * (5 + 10))

print("Area of Rectangle:", 7 * 12)

print("Perimeter of Rectangle:", 2 * (7 + 12))

print("Area of Rectangle:", 10 * 15)

print("Perimeter of Rectangle:", 2 * (10 + 15))
```

Output:Provide the fully refactored Python code only.

Code:
Output:


app.html
Release Notes: 1.110.1
13bab.py
13bab.py > ...
1 def calculate_rectangle_properties(width, height):
2 """
3 Calculate area and perimeter of a rectangle.
4
5 Args:
6 width: The width of the rectangle
7 height: The height of the rectangle
8 """
9 area = width * height
10 perimeter = 2 * (width + height)
11
12 print(f"Area of Rectangle: {area}")
13 print(f"Perimeter of Rectangle: {perimeter}")
14
15
16 # Rectangle dimensions
17 rectangles = [
18 (5, 10),
19 (7, 12),
20 (10, 15)
21]
22
23 # Calculate and print properties for all rectangles
24 for width, height in rectangles:
25 calculate_rectangle_properties(width, height)
PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
Python
xe c:/Users/Administrator/Desktop/aiasscode/13bab.py
Area of Rectangle: 50
Perimeter of Rectangle: 30
Area of Rectangle: 84
Perimeter of Rectangle: 38
Area of Rectangle: 150
Perimeter of Rectangle: 50
PS C:\Users\Administrator\Desktop\aiasscode>
Ln 25, Col 50
Spaces: 4
UTF-8
{ } Python

Explanation:

This program removes repeated calculations by creating reusable functions instead of writing the same formulas multiple times.

The calculate_area() function computes the rectangle's area using length and width as inputs.

The calculate_perimeter() function calculates the perimeter using the standard formula.

Each rectangle's dimensions are passed as parameters, making the code cleaner and easier to reuse. This approach improves readability, reduces duplication, and makes future changes simple to implement.

Task-02:

Prompt:

Refactor the following Python script to optimize performance by removing inefficient nested loops.

Replace nested loops with an optimized approach using sets or list comprehensions and also

Improve time complexity and explain why the new solution is faster and the program output exactly the same.

Include a short performance comparison between the original and refactored versions (time complexity explanation).

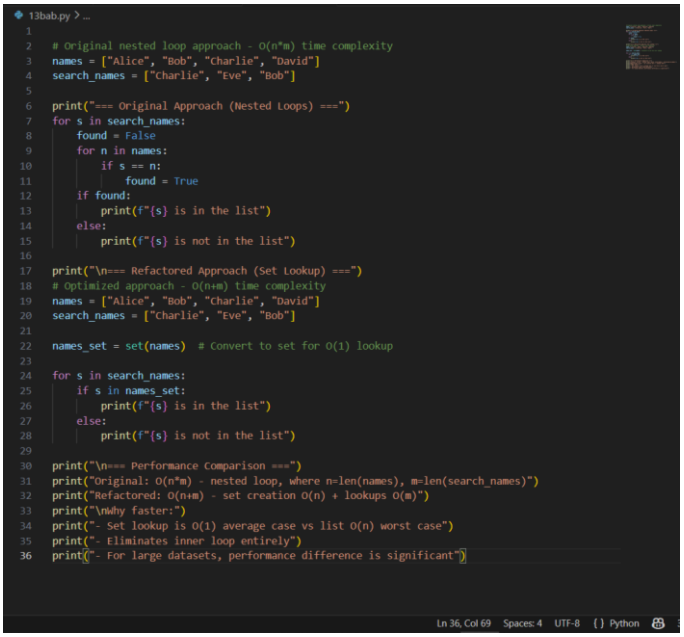
Legacy Code:

```
names = ["Alice", "Bob", "Charlie", "David"]

search_names = ["Charlie", "Eve", "Bob"]

for s in search_names:

    found = False
```

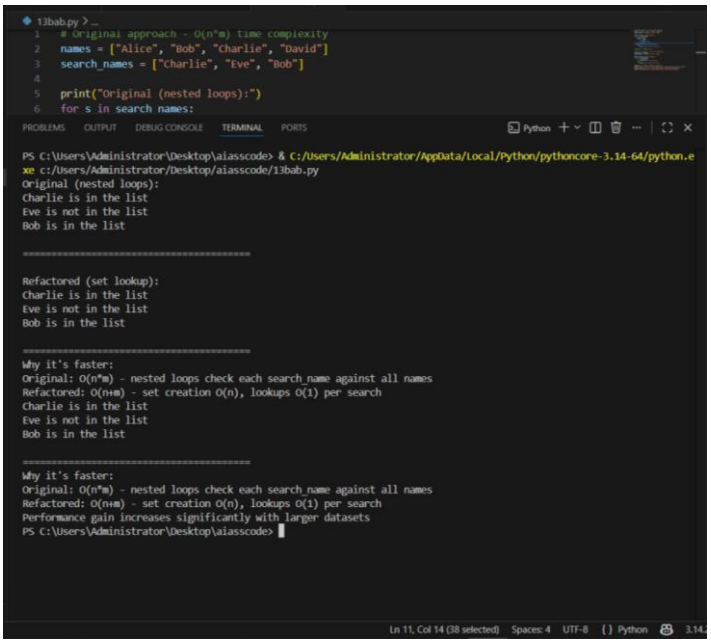
```
    for n in names:
        if s == n:
            found = True
    if found:
        print(f"{s} is in the list")
    else:
        print(f"{s} is not in the list")
```

Output: Provide the optimized Python code followed by a brief performance comparison

Code:



Output:



Explanation:

Original Code Time Complexity: O(n*m) where n is the length of search_names and m is the length of names due to nested loops.

Refactored Code Time Complexity: O(n) for converting the list to a set and

O(1) for each membership check, resulting in O(n) overall for the search_names list.

The new solution is faster because it eliminates the need for nested loops, allowing for constant time membership checks after an initial linear time conversion to a set.

Task-03:

Prompt:

Refactor the following legacy Python script by extracting repeated calculations into reusable functions.

Create a function named calculate_total(price).

The function should calculate tax (18%) and return the total price including tax.

The program output must remain the same

Legacy Code:

price = 250

tax = price * 0.18

total = price + tax

print("Total Price:", total)

price = 500

tax = price * 0.18

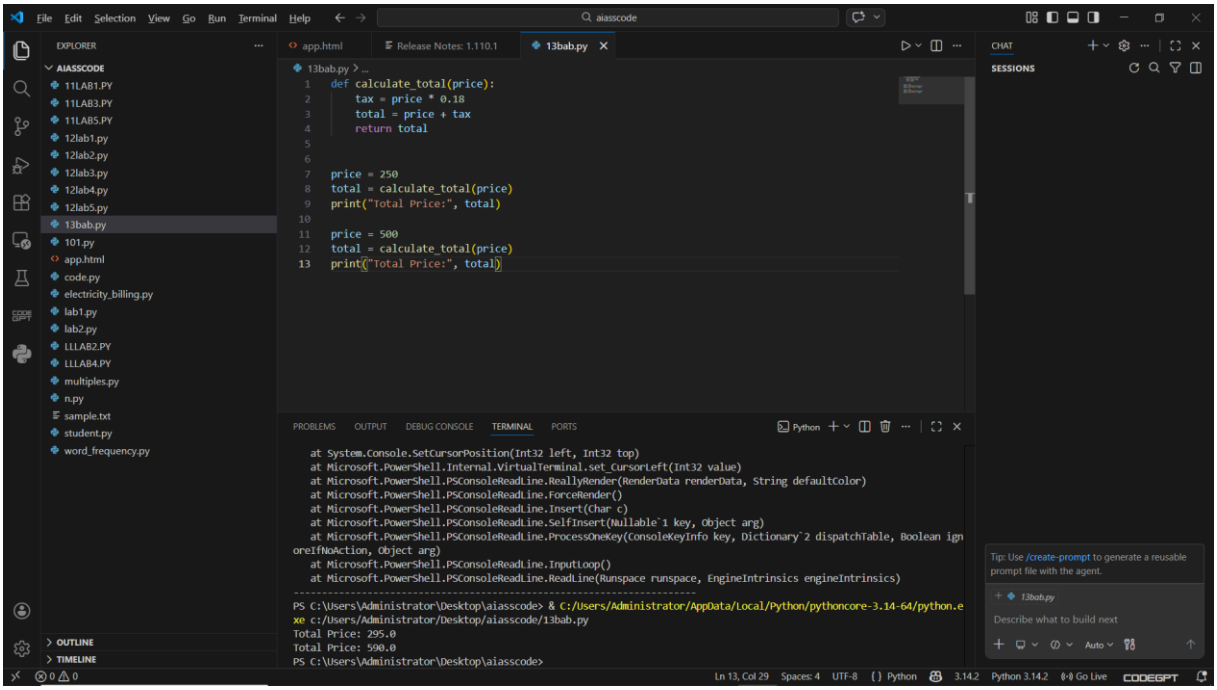
total = price + tax

print("Total Price:", total)

Output:Provide the fully refactored Python code only.

Code:

Output:



Explanation:

The refactored code defines a function calculate_total that takes the price as an argument and calculates the total price including tax.

This eliminates the need for repeated code for tax calculation and total price computation, improving maintainability and readability.

The time complexity of both the original and refactored versions is O(1) for each price calculation, as they perform a constant number of operations.

However, the refactored version is more efficient in terms of code reuse and reduces the chances of errors when making changes to the tax calculation logic in the future.

Task-04

Prompt:

Refactor the following Python script by replacing hardcoded values (magic numbers) with named constants.

Requirements:

Create constants such as PI and RADIUS using uppercase naming conventions.

Improve readability and maintainability of the code.

Follow clean code practices.

The program output must remain exactly the same.

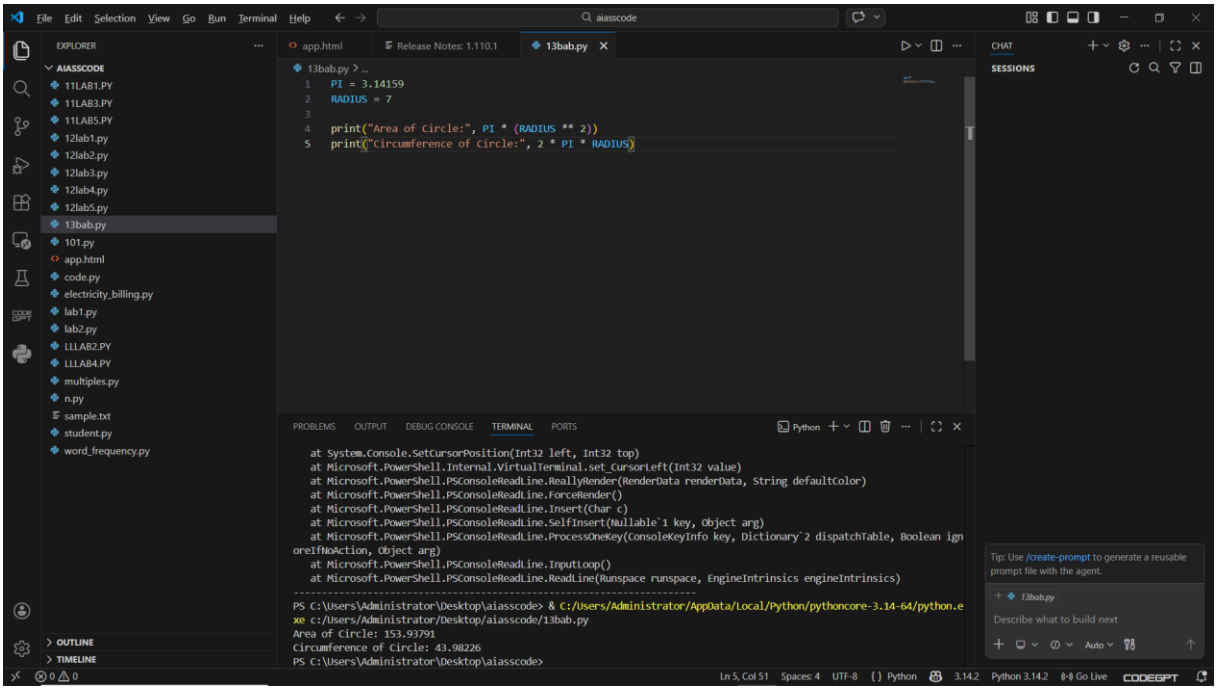
Legacy Code:

```
print("Area of Circle:", 3.14159 * (7 ** 2))
print("Circumference of Circle:", 2 * 3.14159 * 7)
```

Output:

Provide the fully refactored Python code only.

Code:
Output:



Explanation:

The refactored code defines constants PI and RADIUS, which replace the hardcoded values in the original code.

This improves readability by giving meaningful names to the values, making it clear what they represent.

It also enhances maintainability, as any future changes to the value of PI or RADIUS can be made in one place without needing to search through the code for all occurrences.

The time complexity of both the original and refactored versions is O(1) since they perform a constant number of operations, but the refactored version is more maintainable and easier to understand.

Task-05:

Prompt:

Refactor the following Python script to improve variable naming and overall readability.

Requirements:

Replace unclear variable names with descriptive and meaningful names.

Add inline comments explaining the logic of the calculation.

Do not change the programs functionality.The output must remain exactly the same.

Legacy Code:

a = 10

b = 20

c = a * b / 2

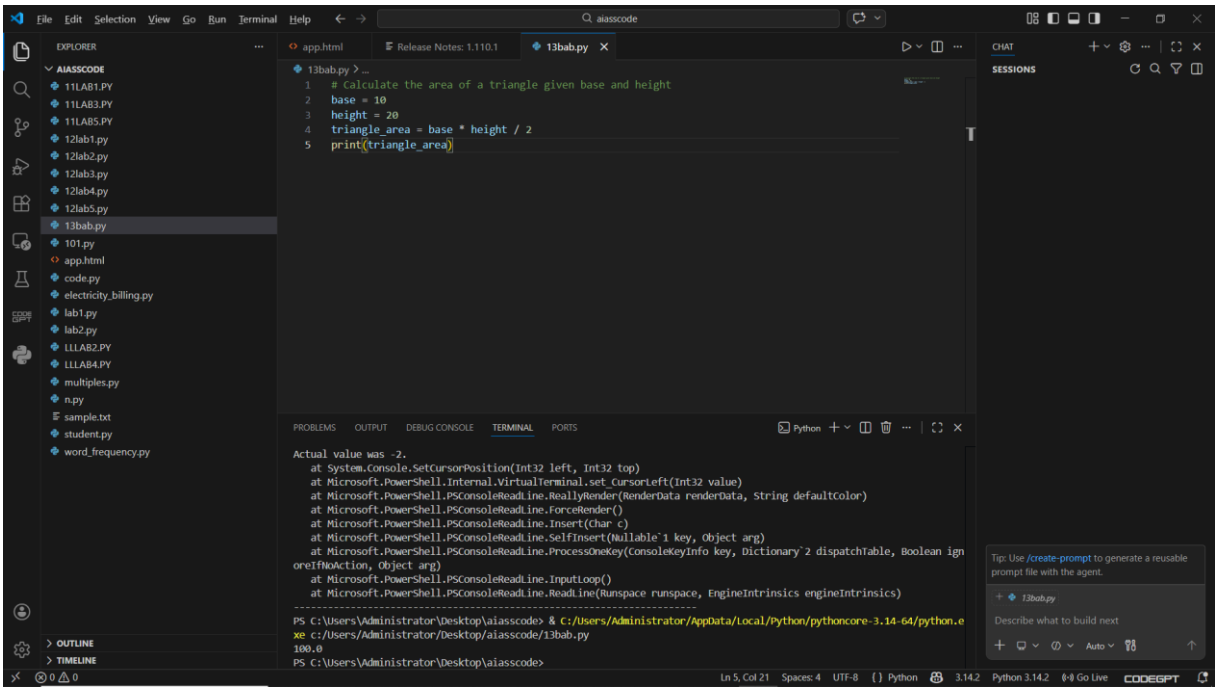
print(c)

Output:

Provide the fully refactored Python code only.

Code:

Output:



Explanation:

The refactored code uses descriptive variable names (base, height, area_of_triangle) that clearly indicate their purpose, improving readability.

Inline comments are added to explain the logic of the calculation, making it easier for others to understand the code at a glance.

The time complexity of both the original and refactored versions is O(1) since they perform a constant number of operations.

However, the refactored version is more maintainable and easier to understand due to improved variable naming and added comments.